

Lenguajes de programación y paradigmas

Estructuras de Datos

Carlos A Delgado S

Pontificia Universidad Javeriana Cali

Julio 2026

Objetivos de aprendizaje I

Al finalizar esta sesión, el estudiante podrá

- Distinguir un intérprete de un compilador y decir qué produce cada uno.
- Reconocer el paradigma de un programa por la forma en que trata la variable.
- Verificar una expresión contra una gramática escrita en BNF.
- Nombrar las etapas y las fases de la compilación, y ejecutarlas a mano con gcc.

Objetivos de aprendizaje II

Objetivos de Aprendizaje del curso involucrados

- OA1: apropiar conocimientos de computación mediante el diseño e implementación de soluciones a problemas pequeños.
- OA3: describir ideas con argumentos precisos y basados en evidencia.

Referencia bibliográfica

R. Sebesta, *Concepts of Programming Languages*, capítulos 1 (*Preliminaries*) y 3 (*Describing Syntax and Semantics*); R. Thareja, *Data Structures Using C*, capítulo 1.

Contenido I

- 1 De la orden al ejecutable
- 2 Lenguajes de programación
- 3 Lenguajes imperativos y lenguajes declarativos
- 4 Estructura de los lenguajes de programación
- 5 Compiladores
- 6 Resumen y referencias

Contenido I

- 1 De la orden al ejecutable
- 2 Lenguajes de programación
- 3 Lenguajes imperativos y lenguajes declarativos
- 4 Estructura de los lenguajes de programación
- 5 Compiladores
- 6 Resumen y referencias

Lo que quedó de la clase pasada I

- Se resolvieron problemas pequeños con condicionales, ciclos, arreglos y funciones.
- La ejecución se siguió con una traza en tabla, iteración por iteración.
- Quedaron fijas las reglas de código limpio: sin break, sin continue, un solo return.

Y se escribió esta orden sin discutirla

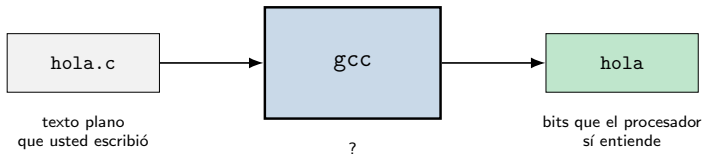
```
$ gcc -Wall -Wextra hola.c -o hola
```

La orden que nadie preguntó I

```
gcc -Wall -Wextra hola.c -o hola
```

- ¿Qué es gcc?
- ¿Qué le hace exactamente a hola.c?
- ¿Por qué hay que pedirle -Wall si el programa ya funciona?

La caja negra I



Adentro no hay un programa: hay varios, y hoy se abren.

-Wall no es decoración I

```
int x;  
  
x = 0;  
if (x = 5) {  
    printf("entro al if aunque x valia 0\n");  
}
```

Con -Wall -Wextra

```
warn_asignacion.c:8:9: warning: suggest parentheses around  
assignment used as truth value [-Wparentheses]
```

- Sin banderas, gcc compila callado: ni una línea de salida.
- Es el = contra == de la clase pasada. El aviso nace en una fase concreta del compilador, y al final de la sesión se sabrá en cuál.

Contenido I

- 1 De la orden al ejecutable
- 2 Lenguajes de programación
- 3 Lenguajes imperativos y lenguajes declarativos
- 4 Estructura de los lenguajes de programación
- 5 Compiladores
- 6 Resumen y referencias

Dígale esto al computador I

Sume los números de la lista y muéstreme el resultado.

¿Por qué el computador no puede ejecutar esa frase tal como está?

Porque admite dos lecturas I

“Los números de la lista”

¿La lista entera, o solo hasta el primer negativo? La frase no lo dice y las dos lecturas son legítimas.

“Muéstreme el resultado”

¿Escribirlo en la pantalla, o devolvérselo a quien preguntó para que siga calculando? Tampoco lo dice.

- El español vive bien con esa holgura: quien escucha completa lo que falta.
- Un lenguaje de programación no puede permitírsela. Cada construcción tiene que significar una sola cosa.

Qué es un lenguaje de programación I

Definición

Los algoritmos se desarrollan e implementan mediante lenguajes de programación. Un lenguaje de programación es un mecanismo de comunicación compuesto por un vocabulario y un conjunto de reglas gramaticales. Su propósito es ordenarle al computador que realice una tarea específica.

- El vocabulario son las palabras admitidas: `int`, `while`, `return`, los nombres que usted inventa, los operadores.
- Las reglas gramaticales dicen en qué orden pueden aparecer.
- Falta algo que la definición no menciona y esta sesión sí trata: el significado de cada construcción.

Sebesta, *Concepts of Programming Languages*, cap. 1.

El mismo cálculo, al nivel de la máquina I

El ciclo del factorial, tal como lo dejó gcc -S en factorial.s:

```
.L3:
    movl    -12(%rbp), %eax
    cltq
    movq    -8(%rbp), %rdx
    imulq   %rdx, %rax
    movq    %rax, -8(%rbp)
    addl    $1, -12(%rbp)
.L2:
    movl    -12(%rbp), %eax
    cmpl    -20(%rbp), %eax
    jle     .L3
```

- No hay variables con nombre: hay registros (%eax,%rdx) y desplazamientos respecto de %rbp.
- No hay while: hay una comparación y un salto.

El mismo cálculo, en C y en Python I

```
long factorial(int n) {  
    long resultado;  
    int i;  
  
    resultado = 1;  
    i = 2;  
    while (i <= n) {  
        resultado = resultado * i;  
        i = i + 1;  
    }  
    return resultado;  
}
```

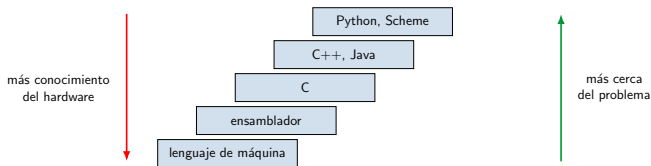
```
from math import prod  
  
print(prod(range(2, 6)))
```

¿Cuál escribiría usted a las once de la noche, y cuál necesita saber qué es %rbp?

El nivel de un lenguaje I

Definición

Un lenguaje puede hacer el desarrollo de programas más fácil o más difícil según el nivel de abstracción que requiera y la cantidad de conocimiento del funcionamiento interno del computador que exija. Entre más familiar sea el lenguaje para expresar el problema, más alto es su nivel.

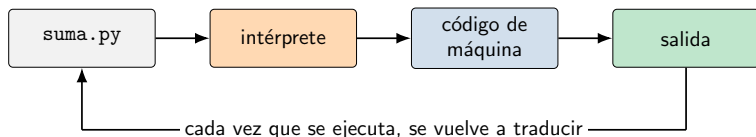


Nadie ejecuta C I

- El procesador solo entiende secuencias de bits.
- El archivo `factorial.c` es texto plano: se abre con cualquier editor y se lee.

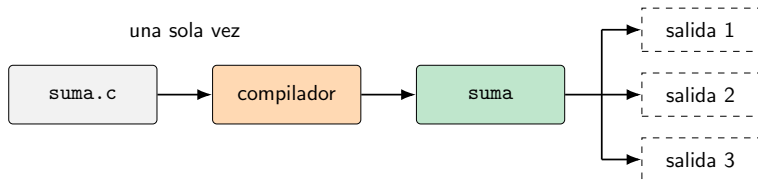
¿Quién hace la traducción, y sobre todo, *cuándo* la hace?

El intérprete I



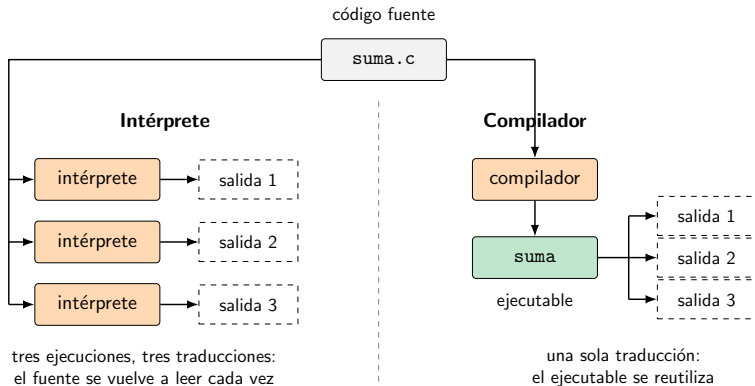
- El traductor procesa el programa línea a línea, en cada ejecución, y produce el programa en el lenguaje que la máquina entiende.
- Si se quiere correr mil veces, se traduce mil veces.

El compilador I



- El compilador traduce el programa una vez y produce otro programa, que suele llamarse ejecutable.
- Para volver a correrlo se reutiliza la traducción que se hizo originalmente.

Los dos, lado a lado I



Las cinco diferencias I

	Intérprete	Compilador
Cuándo traduce	en cada ejecución	una sola vez
Qué produce	nada que quede en disco	un ejecutable
Al ejecutar de nuevo	vuelve a traducir	reutiliza la traducción
Cuándo aparecen los errores	al llegar a la línea	antes de correr
Qué debe estar instalado	el intérprete	nada, solo el ejecutable

Mil veces el mismo ciclo I

```
long suma;  
long i;  
  
suma = 0;  
i = 0;  
while (i < 50000000) {  
    suma = suma + i;  
    i = i + 1;  
}  
printf("%ld\n", suma);
```

```
suma = 0  
i = 0  
while i < 50000000:  
    suma = suma + i  
    i = i + 1  
print(suma)
```

Los dos imprimen 1249999975000000. ¿Cuánto se separan los tiempos?

La medición I

Versión	Cómo se ejecuta	Tiempo
C, gcc -O0	compilada	0,065 s
Python 3	interpretada	9,295 s

- Unas 140 veces, medidas en el equipo de la clase.
- Lo medido es la implementación, no la elegancia del lenguaje: nada impide escribir un compilador de Python.
- Java y Python son híbridos: compilan a un código intermedio y después lo interpretan.

Sebesta, cap. 1, sección *Implementation Methods*.

Contenido I

- 1 De la orden al ejecutable
- 2 Lenguajes de programación
- 3 Lenguajes imperativos y lenguajes declarativos**
- 4 Estructura de los lenguajes de programación
- 5 Compiladores
- 6 Resumen y referencias

Dos programas I

```
long factorial(int n) {  
    long resultado;  
    int i;  
  
    resultado = 1;  
    i = 2;  
    while (i <= n) {  
        resultado = resultado * i;  
        i = i + 1;  
    }  
    return resultado;  
}
```

```
(define (factorial n)  
  (if (= n 0)  
      1  
      (* n (factorial (- n 1)))))
```

¿Hacen lo mismo? ¿En qué se diferencian? I

```
resultado = 1;
i = 2;
while (i <= n) {
    resultado = resultado * i;
    i = i + 1;
}
```

```
(if (= n 0)
  1
  (* n (factorial (- n 1))))
```

Iguales en el qué, distintos en el cómo I

- Los dos calculan el factorial de un entero. En ese sentido son iguales: hacen lo mismo, y para $n = 5$ los dos responden 120.
- Al leer instrucción por instrucción aparecen diferencias grandes, y esas diferencias corresponden al paradigma de programación empleado.

Paradigma de programación

Un paradigma determina aspectos de la programación tales como los conceptos algorítmicos empleados para crear programas, el concepto de variable, la noción de tipo de dato y las estructuras de datos, y el uso de la memoria. Existe una gran variedad de paradigmas, pero la mayoría son variaciones o extensiones de dos: el declarativo y el imperativo.

Declarativo: QUÉ se calcula I

Paradigma declarativo

En los lenguajes declarativos el programador se concentra en especificar QUÉ desea calcular, sin especificar el cómo. El cómo son las operaciones de bajo nivel que deben realizarse, su orden y todo lo relacionado con la manipulación de memoria.

- En Scheme nadie dijo por dónde empezar ni en qué orden multiplicar: se dijo qué es el factorial y el lenguaje se encargó del resto.

La variable del declarativo I

Definición

En el paradigma declarativo la variable se usa en el sentido de las funciones matemáticas: una entidad que se sustituye por un valor pero cuyo valor nunca cambia. No hay estado explícito. Por lo mismo, no hay asociación directa entre la variable, el espacio que ocupa en memoria y la forma de manipular ese espacio.

- **Lenguajes:** Lisp y muchos de sus dialectos, como Scheme; Mozart; Haskell; Erlang; Maude.
- **Variaciones:** paradigma funcional y paradigma de programación lógica.

Scheme: el factorial funcional I

```
; Paradigma funcional: se dice QUE es el factorial,  
; no como calcularlo  
(define (factorial n)  
  (if (= n 0)  
      1  
      (* n (factorial (- n 1)))))  
  
(display (factorial 5))  
(newline)
```

- No hay asignación y no hay ciclo: hay una definición que se nombra a sí misma.
- Ejecutado con guile -s factorial.scm, imprime 120.

Evaluar (factorial 4) a mano I

Sustitución paso a paso

Paso	Expresión
0	(factorial 4)
1	(* 4 (factorial 3))
2	(* 4 (* 3 (factorial 2)))
3	(* 4 (* 3 (* 2 (factorial 1))))
4	(* 4 (* 3 (* 2 (* 1 1))))
5	24

- Ninguna casilla de memoria cambió de contenido. Lo que ocurrió fue que la expresión se reescribió.
- n nunca “pasó” de 4 a 3: cada llamada tiene su propio n y ninguno se modifica.

Prolog: el factorial lógico I

```
% Paradigma logico: se declaran los hechos y las reglas  
% que definen la relacion  
factorial(0, 1).  
factorial(N, F) :-  
    N > 0,  
    M is N - 1,  
    factorial(M, G),  
    F is N * G.
```

- La primera línea es un hecho: el factorial de 0 es 1.
- La segunda es una regla: F es el factorial de N si se cumplen las cuatro submetas de la derecha.

Cómo responde la consulta I

```
?- factorial(4, F).  
    F = 24.
```

- El motor unifica la consulta con la cabeza de la regla y liga $N = 4$.
- Desciende resolviendo submetas hasta tropezar con el hecho `factorial(0, 1)`.
- Al regresar liga $F = 24$.
- La repetición no es un ciclo ni exactamente una llamada: es unificación con retroceso.

Sin verificar en esta máquina

El equipo de la clase no tiene intérprete de Prolog instalado. El archivo `factorial.pl` va como ilustración y deja la consulta comentada; ese $F = 24$ no se ejecutó aquí.

Imperativo: CÓMO se calcula I

Paradigma imperativo

En contraste con el declarativo, en los lenguajes imperativos el programador se concentra en especificar CÓMO realizar el cálculo: las operaciones de bajo nivel que deben realizarse, su orden y todo lo relacionado con la manipulación de memoria.

- En la versión en C se dijo dónde empezar (`resultado = 1`), en qué orden multiplicar y cuándo parar.

La variable del imperativo I

Definición

En el paradigma imperativo la variable es una referencia, un nombre o un alias de un espacio de memoria. Está asociada a un tamaño que determina qué valores puede almacenar, y su valor cambia durante la ejecución del programa: hay estado explícito. Un programa imperativo es una secuencia de pasos que dice qué espacios de memoria usar, cómo varían los valores guardados en ellos y cómo se construye progresivamente un resultado.

- **Lenguajes:** C/C++, Python, Java, Mozart, Common Lisp.
- **Variaciones:** paradigma orientado a objetos y paradigma orientado a agentes.

C: el factorial imperativo I

```
/* Paradigma imperativo: se dice COMO calcular el factorial */  
long factorial(int n) {  
    long resultado;  
    int i;  
  
    resultado = 1;  
    i = 2;  
    while (i <= n) {  
        resultado = resultado * i;  /* la variable cambia de valor */  
        i = i + 1;  
    }  
    return resultado;  
}
```

- Compila sin advertencias con gcc -Wall -Wextra y factorial(5) da 120.

Traza de factorial(4) en C I

Las dos casillas, iteración por iteración

Iteración	$i \leq 4?$	resultado	i
inicio	—	1	2
1	sí	2	3
2	sí	6	4
3	sí	24	5
4	no	se sale del ciclo	

- Compare con la traza de Scheme: allá la expresión crecía, acá dos casillas se sobrescriben cuatro veces.
- El resultado es el mismo, 24. El camino no.

C++: el factorial orientado a objetos I

```
class Factorial {  
private:  
    long resultado;  
  
public:  
    Factorial() {  
        resultado = 1;  
    }  
  
    long calcular(int n) {  
        int i;  
  
        resultado = 1;  
        i = 2;  
        while (i <= n) {  
            resultado = resultado * i;  
            i = i + 1;  
        }  
        return resultado;  
    }  
};
```

Lo que cambió y lo que no l

```
int main() {  
    Factorial f;  
  
    std::cout << f.calcular(5) << std::endl;  
    return 0;  
}
```

- El ciclo es idéntico al de C: se sigue diciendo el cómo.
- Lo que cambió es dónde vive el estado: resultado dejó de ser una variable local y pasó a ser un atributo del objeto f.
- Por eso la orientación a objetos es una variación del paradigma imperativo, no un paradigma aparte.
- Ejecutado con `g++ -Wall -Wextra`, imprime 120.

Los cuatro, en una tabla I

Paradigma	Lenguaje	Idea central	La variable
Imperativo	C	una secuencia de pasos	casilla de memoria que cambia
Orientado a objetos	C++	los pasos y el estado viven dentro de un objeto	atributo del objeto
Funcional	Scheme	la función se define a sí misma	nombre que se sustituye por un valor
Lógico	Prolog	hechos y reglas; el motor busca la respuesta	incógnita que el motor liga

- Los tres primeros se ejecutaron en el equipo de la clase y los tres imprimieron 120.

Contenido I

- 1 De la orden al ejecutable
- 2 Lenguajes de programación
- 3 Lenguajes imperativos y lenguajes declarativos
- 4 Estructura de los lenguajes de programación
- 5 Compiladores
- 6 Resumen y referencias

Características de los lenguajes de alto nivel I

Los lenguajes de alto nivel comparten cinco rasgos

1 Datos y tipos de datos.

2 Operaciones primitivas.

3 Secuencias de control.

4 Almacenamiento.

5 Interacción con el ambiente.

$\sim$ int $x \in \mathbb{Z}$
32 bits $[-2^{31}, 2^{31}-1]$
unsigned int $[0, 2^{32}-1]$
long $[-2^{63}, 2^{63}-1]$

- En C ya se vieron los cinco: int y long; + y *; if y while; los arreglos; printf y scanf.
- El resto del semestre profundiza sobre todo en el primero y el cuarto.

Sintaxis y semántica I

Definición

Al igual que cualquier lenguaje, incluido el español, los lenguajes de programación tienen una sintaxis y una semántica. La sintaxis es la forma en que los programas se escriben; la semántica es el significado que se le da a esas construcciones sintácticas.

- Cada lenguaje tiene una sintaxis y una semántica propias.

La misma idea, dos sintaxis I

Una lista de diez reales. En Pascal:

```
var V: array [1..10] of real;
```

En C/C++:

```
double V[10];
```

- La semántica es prácticamente la misma: diez casillas contiguas de números con decimales.
- La sintaxis no se parece en nada. Pascal numera de 1 a 10 y lo dice; C escribe el tamaño y numera de 0 a 9 sin decirlo.

Sintaxis: los tokens I

Definición

La sintaxis de un lenguaje está definida como la escogencia y organización de varios elementos sintácticos básicos. Esos elementos, llamados tokens, pueden ser caracteres, identificadores, operadores, palabras reservadas, comentarios, espacios en blanco y delimitadores. La especificación léxica del lenguaje es la definición de estos elementos.

Una línea, sus tokens I

```
resultado = resultado * i;
```

Token	Categoría
resultado	identificador
=	operador
resultado	identificador
*	operador
i	identificador
;	delimitador

- Los espacios separan los tokens y después sobran: no aparecen en la tabla.

De los tokens a la gramática I

Definición

Los tokens conforman expresiones, declaraciones y, en general, la estructura de un programa. La organización de esos elementos en categorías sintácticas determina la gramática del lenguaje. La gramática consta de un conjunto de definiciones, llamadas reglas o producciones, que especifican el orden particular en que deben estar ubicados los elementos para que un programa esté bien escrito.

- La forma más usada de especificar una gramática es la BNF (*Backus-Naur Form*), desarrollada por J. Backus en 1960.
- La BNF define un lenguaje de manera directa: cada categoría se escribe en términos de otras y de tokens.

BNF: qué es un identificador I

```
<identificador> ::= <letra>
                  | "_"
                  | <identificador> ( <letra> | "_" )
                  | <identificador> <digito>

    <letra> ::= <mayuscula> | <minuscula>
<minuscula> ::= "a" | "b" | ... | "z"
<mayuscula> ::= "A" | "B" | ... | "Z"
    <digito> ::= "0" | "1" | ... | "9"
```

- Lo que está entre ángulos es una categoría sintáctica; lo que está entre comillas es un token literal.
- La barra vertical separa alternativas.
- La primera regla se nombra a sí misma: por eso un identificador puede tener el largo que sea.

Java Syntax Specification

Programs

<compilation unit> ::= <package declaration>? <import declarations>? <type declarations>?

Declarations

<package declaration> ::= package <package name> ;

<import declarations> ::= <import declaration> | <import declarations> <import declaration>

<import declaration> ::= <single type import declaration> | <type import on demand declaration>

<single type import declaration> ::= import <type name> ;

*<type import on demand declaration> ::= import <package name> . * ;*

<type declarations> ::= <type declaration> | <type declarations> <type declaration>

<type declaration> ::= <class declaration> | <interface declaration> | ;

<class declaration> ::= <class modifiers>? class <identifier> <super>? <interfaces>? <class body>

<class modifiers> ::= <class modifier> | <class modifiers> <class modifier>

<https://cs.au.dk/~amoeller/RegAut/JavaBNF.html>

¿Por qué 2suma no compila? I

La BNF ya lo respondió

Ninguna de las cuatro alternativas de identificador empieza por un dígito: la primera arranca con una letra, la segunda con el guion bajo, y las dos últimas arrancan con otro identificador, que a su vez tiene que empezar con letra o guion bajo.

- suma2 sí es válido: es la cuarta alternativa, identificador seguido de dígito.
- El compilador no tiene una lista de nombres prohibidos. Tiene esta gramática.

BNF: la asignación y la expresión aritmética I

```
<asignacion> ::= <identificador> "=" <exp_aritmetica> ";"  
  
<exp_aritmetica> ::= <entero>  
                    | <identificador>  
                    | <exp_aritmetica> "+" <exp_aritmetica>  
                    | <exp_aritmetica> "-" <exp_aritmetica>  
                    | <exp_aritmetica> "*" <exp_aritmetica>  
                    | <exp_aritmetica> "/" <exp_aritmetica>  
  
    <entero> ::= <digito_sin_cero> <digito>* | "0"  
<digito_sin_cero> ::= "1" | ... | "9"
```

- Con estas cuatro producciones queda descrita cualquier asignación con enteros y las cuatro operaciones básicas.

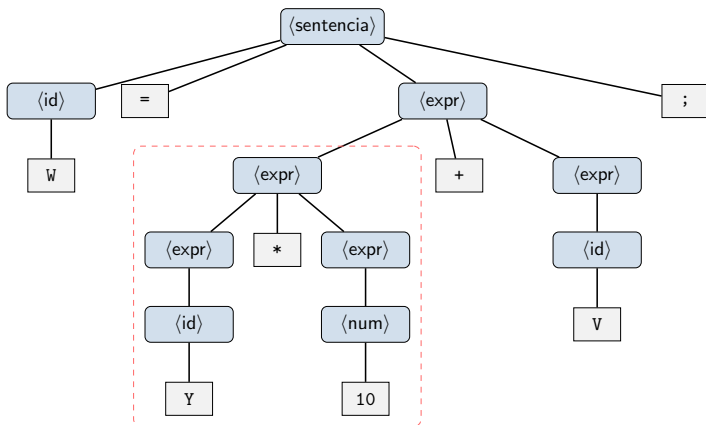
¿Está bien escrita? I

W = Y * 10 + V;

$W = V + Y * 10$

Para asegurar que una expresión está bien escrita hay que seguir la BNF de forma estricta.

Sí lo está: el árbol lo sostiene I



Y * 10 cuelga por debajo de la suma: se reduce primero

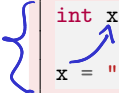
Definición

Además de una sintaxis, cada lenguaje tiene asociada una semántica. La semántica determina las acciones que se llevan a cabo en un algoritmo, el resultado y el procedimiento realizado en cada operación. También fija ciertas propiedades asociadas a los tipos y valores de las variables, el comportamiento de las estructuras de control y de repetición, y otras propiedades ligadas al paradigma en el que se basa el lenguaje.

- El árbol de la lámina anterior dice que $Y * 10$ cuelga por debajo de la suma. La semántica es la que convierte esa forma en una orden: multiplique primero, sume después.

La sintaxis no basta I

Bien escrito, sin sentido



```
int x;  
x = "cinco";
```

- Identificador, =, expresión, punto y coma: la gramática está contenta.
- La semántica no: `x` es un espacio para un entero y lo que se le quiere meter es la dirección de una cadena.
- Ese desacuerdo lo detecta una fase concreta del compilador. La siguiente sección la nombra.

Contenido I

- 1 De la orden al ejecutable
- 2 Lenguajes de programación
- 3 Lenguajes imperativos y lenguajes declarativos
- 4 Estructura de los lenguajes de programación
- 5 Compiladores**
- 6 Resumen y referencias

El programa que se va a seguir I

```
/* Programa minimo para seguir el proceso de compilacion */  
#include <stdio.h>  
  
#define SALUDO "Hola, Estructuras de Datos"  
  
int main(void) {  
    printf("%s\n", SALUDO);  
    return 0;  
}
```

- Nueve líneas. Un include, un define y un printf.
- La caja negra de la primera sección se abre con este archivo.

Cuatro etapas, no una l

Definición

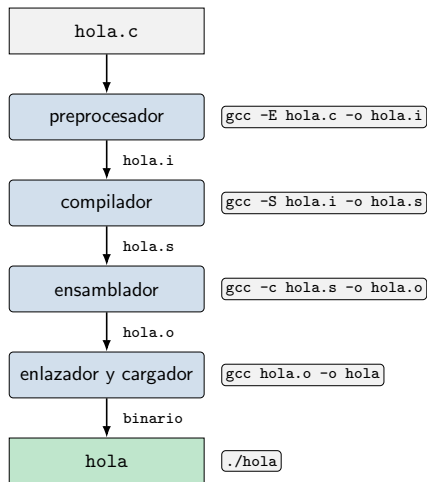
Para hacer la traducción del programa deben superarse varias etapas, que pueden verse como los componentes intermedios del compilador: el preprocesamiento, el compilador propiamente dicho, el ensamblador, y los enlazadores y cargadores.

- Los programas escritos en un lenguaje de programación se ejecutan mediante intérpretes o compiladores, que convierten el código fuente en un código objetivo: código intermedio o código de máquina.

C/C++

JAVA

El sistema de procesamiento I



Etapa 1: preprocesamiento I

Qué hace

El código fuente puede estar repartido en varios archivos o módulos, y puede haber macros y extensiones necesarias para el programa. Todos esos componentes se recolectan para tener un único programa fuente.

```
$ gcc -E hola.c -o hola.i
```

- `hola.c` tiene 9 líneas. `hola.i` tiene 847.
- Las 838 líneas nuevas son `stdio.h` entero, pegado donde estaba el `include`.

Lo que quedó al final de hola.i |

```
extern int printf (const char *__restrict __format, ...);

/* ... 350 lineas mas de stdio.h ... */

int main(void) {
    printf("%s\n", "Hola, Estructuras de Datos");
    return 0;
}
```

- El define desapareció: donde decía SALUDO ahora está la cadena literal.
- El preprocesador no entiende C. Solo pega texto y reemplaza texto.

Etapa 2: el compilador propiamente dicho I

```
$ gcc -S hola.i -o hola.s
```

```
.LC0:
    .string "Hola, Estructuras de Datos"
    .text
    .globl main
main:
    pushq    %rbp
    movq     %rsp, %rbp
    leaq     .LC0(%rip), %rax
    movq     %rax, %rdi
    call     puts@PLT
    movl     $0, %eax
    popq     %rbp
    ret
```

- 847 líneas entraron; 28 salieron.
- La lámina omite las directivas `.cfi`, que no son instrucciones del procesador: le dicen al depurador cómo reconstruir la pila.

El código ensamblador I

Definición

El código ensamblador es una versión menos abstracta del código de máquina, que es el que entiende el procesador. Trabaja directamente con direcciones de memoria de la RAM, los registros del procesador, la pila del programa y las interrupciones del sistema operativo. Cada procesador tiene un conjunto de instrucciones que constituye su lenguaje ensamblador.

- Este `hola.s` es de x86-64. En un teléfono, con procesador ARM, el mismo `hola.c` produce otras instrucciones.
- Detalle curioso: el fuente decía `printf` y el ensamblador dice `puts`. Como la cadena no lleva formatos que resolver, `gcc` cambió la llamada por una más barata.

Etapa 3: el ensamblador I

```
$ gcc -c hola.s -o hola.o
$ file hola.o
hola.o: ELF 64-bit LSB relocatable, x86-64, version 1
        (SYSV), not stripped
```

- Aquí el texto se acaba: hola.o ya es binario.
- *Relocatable* quiere decir que todavía no tiene decidido en qué direcciones va a vivir.

Lo que sabe y lo que no sabe hola.o |

```
$ nm hola.o
0000000000000000 T main
                  U puts
```

- La T de main dice que ese símbolo está definido aquí, en la sección de texto.
- La U de puts dice *undefined*: el objeto usa esa función pero no la tiene. Alguien la tendrá que poner.

Por qué esto importa

Una vez pasado el ensamblador, el código objeto puede usarse como librería para otros programas. Los componentes intermedios del compilador son separables: el proceso puede hacerse paso a paso, tomando cada componente como una aplicación aparte, que es exactamente lo que se acaba de hacer.

Etapa 4: enlazadores y cargadores I

Definición

Los enlazadores y cargadores cargan y enlazan el código intermedio y las librerías ya creadas al programa que se está traduciendo: archivos `.dll` en Windows, `.so` y `.a` en Linux.

```
$ gcc hola.o -o hola
$ ./hola
Hola, Estructuras de Datos
```

- El enlazador buscó `puts` en la librería estándar de C y resolvió la U.
- Nueve líneas de texto, cuatro órdenes, un ejecutable.

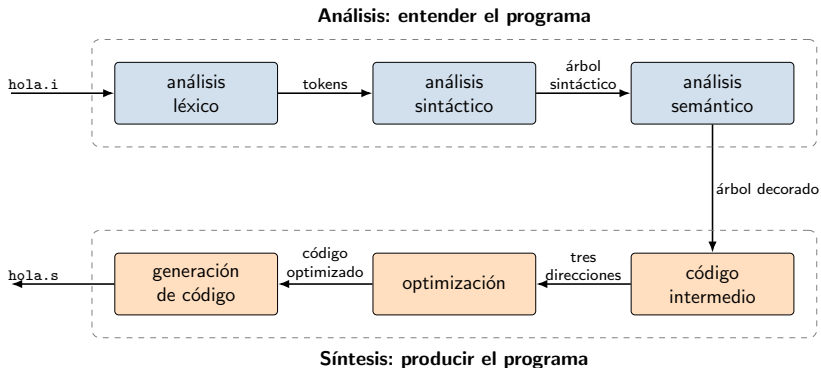
Adentro del compilador: seis fases I

Definición

Conceptualmente, el proceso de transformar el programa fuente en un programa ensamblador se descompone en seis fases: análisis léxico, análisis sintáctico, análisis semántico, generación de código intermedio, optimización y generación de código.

- Las tres primeras entienden el programa; las tres últimas lo producen.
- Todas ocurren dentro de la segunda etapa, la que convierte `hola.i` en `hola.s`.

Las seis fases I



Fase 1: análisis léxico I

Qué hace

Escanea los caracteres del programa en busca de símbolos que no hacen parte del alfabeto del lenguaje. Cuando un grupo de caracteres que conforma un elemento sintáctico básico queda escaneado y aprobado, se pasa como un token al analizador sintáctico. Los espacios en blanco que separan los tokens se eliminan en esta fase.

- Es la fase que produjo la tabla de tokens de resultado = resultado * i;.
- Trabaja con caracteres y no sabe nada de estructura: para ella, ; ; ; son tres tokens perfectamente respetables.

Fase 2: análisis sintáctico I

Qué hace

El analizador sintáctico crea un árbol con cada uno de los tokens para comparar las expresiones con la BNF del lenguaje.

- Es el árbol de $W = Y * 10 + V$; que se armó en la sección anterior, solo que aquí lo arma un programa.
- Si los tokens no se dejan acomodar en ningún árbol válido, el programa no respeta la gramática y la compilación se detiene.

Fase 3: análisis semántico I

Qué hace

Revisa el programa fuente buscando errores de tipos de datos, de control de flujo y de unicidad.

- **Tipos:** que los operadores se apliquen a operandos correctos y que los valores asignados quepan en los espacios de memoria reservados.
- **Control de flujo:** que los comandos que rompen el flujo, como `break`, tengan a dónde transferirlo.
- **Unicidad:** que no haya variables ni etiquetas distintas con el mismo identificador.
- Aquí muere el `x = "cinco"` de la sección anterior, y de aquí sale la advertencia de `if (x = 5)`.

Fases 4, 5 y 6: la síntesis I

Código intermedio

Superadas las fases de análisis, algunos compiladores generan un código intermedio. Ese código le sirve al compilador para decidir el orden de operación de las líneas y para generar nombres temporales que guarden el valor calculado por cada instrucción.

Optimización y generación

La fase de optimización intenta mejorar el código intermedio para obtener mejores resultados en rendimiento. La última fase es la generación del código objetivo, que usualmente consiste en código ensamblador.

- El `printf` que salió convertido en `puts` es una decisión de estas fases.

Cuatro programas rotos I

```
int x = 5 @ 3;      /* err_lexico.c    */

int x = 5           /* err_sintactico.c */
printf("%d\n", x);

int x;
x = "cinco";        /* err_semantico.c  */

int poder_de_butters(int n); /* err_enlace.c: se declara,
                             nunca se define */
printf("%d\n", poder_de_butters(3));
```

¿En qué fase muere cada uno?

Léxico y sintáctico I

Análisis léxico

```
err_lexico.c:4:15: error: stray '@' in program
  4 |         int x = 5 @ 3;
    |                     ^
```

stray es “suelto, extraviado”. El escáner encontró un carácter que no pertenece al alfabeto de C y ni siquiera intentó darle sentido.

Análisis sintáctico

```
err_sintactico.c:5:5: error: expected ',', or ';' before 'printf'
  5 |         printf("%d\n", x);
    |         ~~~~~
```

Todos los tokens son legales. Lo que falla es el orden: la gramática exige un punto y coma donde apareció un identificador.

Análisis semántico

```
err_semantico.c:6:7: error: assignment to 'int' from 'char *'  
makes integer from pointer without a cast [-Wint-conversion]  
  6 |      x = "cinco";  
    |      ^
```

- La forma es impecable: identificador, =, expresión, punto y coma.
- El compilador tuvo que saber que `x` es `int` y que `"cinco"` es `char *`. Eso ya no es gramática: es significado.
- Fíjese en el error de la fase anterior. Ahí gcc citaba tokens; acá cita tipos.

El que no muere en ninguna fase I

```
$ gcc -Wall -Wextra -c err_enlace.c -o err_enlace.o  
$ ls -l err_enlace.o  
-rw-r--r-- 1576 err_enlace.o
```

- Las seis fases pasaron sin una sola queja y el objeto quedó escrito: 1576 bytes.
- Y sin embargo el programa no existe todavía.

Y aun así, no enlaza l

Al pedir el ejecutable

```
$ gcc err_enlace.o -o err_enlace
/usr/bin/ld: err_enlace.o: in function 'main':
err_enlace.c:(.text+0xa): undefined reference to
'poder_de_butters'
collect2: error: ld returned 1 exit status
```

- Quien se queja ya no es gcc: es ld, el enlazador, en la cuarta etapa.
- La declaración le prometió al compilador que la función existía en alguna parte. El compilador creyó. El enlazador fue a buscarla y no estaba.

El mapa completo I

Síntoma	Quién se queja	Dónde
stray '@' in program	analizador léxico	fase 1
expected ';' before ...	analizador sintáctico	fase 2
makes integer from pointer	analizador semántico	fase 3
undefined reference to ...	ld	etapa 4, enlazado

- Leer el nombre de quien se queja ahorra la mitad de la depuración.

Contenido I

- 1 De la orden al ejecutable
- 2 Lenguajes de programación
- 3 Lenguajes imperativos y lenguajes declarativos
- 4 Estructura de los lenguajes de programación
- 5 Compiladores
- 6 Resumen y referencias

Resumen I

Ideas clave

- Un lenguaje de programación es un vocabulario más un conjunto de reglas gramaticales, y sirve para ordenarle una tarea al computador.
- El intérprete traduce en cada ejecución; el compilador traduce una vez y deja un ejecutable reutilizable.
- El paradigma fija el concepto de variable: en el declarativo se sustituye por un valor y no cambia; en el imperativo es un espacio de memoria con estado.
- La sintaxis dice cómo se escriben los programas y se especifica en BNF; la semántica dice qué significan.
- Compilar son cuatro etapas separables, y dentro de la segunda hay seis fases. Cada error tiene su fase.

Para la próxima sesión I

- Tome `hola.c` y córralo con las cuatro órdenes por separado: `-E`, `-S`, `-c` y el enlazado. Mire cuántas líneas tiene cada archivo intermedio.
- Cambie `printf("%s\n", SALUDO)` por `printf("%s y %d\n", SALUDO, 7)` y vuelva a mirar `hola.s`. ¿Sigue apareciendo `puts`?
- Escriba tres programas rotos, uno por fase de análisis, y prediga el mensaje de `gcc` antes de compilarlos.

Referencias I

-  R. Sebesta. *Concepts of Programming Languages*. Pearson, 2015. Capítulos 1 (*Preliminaries*) y 3 (*Describing Syntax and Semantics*).
-  R. Thareja. *Data Structures Using C*. Oxford University Press, 2018. Capítulo 1 (*Introduction to C*).
-  C. A. Ramírez Restrepo. *Estructuras de Datos: Paradigmas de Programación*. Pontificia Universidad Javeriana Cali, 2025-2.

<https://www.carlosramirez.info/teaching/estructuras-de-datos/2025-2>